Experiment 1

Low Code vs Traditional Application Development

1. Aim

To compare low-code and traditional application development by analysing how a Gym Membership Management application (members, plans, subscriptions, payments, attendance) would be built using each approach, and to justify the choice of Zoho Creator for this project.

2. Introduction

Traditional development means hand-writing the front end, back end, database schema, authentication and deployment pipeline using general-purpose languages and frameworks (for example React, Node.js and MySQL). It offers complete control but needs a skilled team and a long delivery cycle.

Low-code development uses a visual builder with drag-and-drop forms, built-in data storage, workflow tools and a small scripting layer for custom logic. **Zoho Creator** is one such platform; its scripting language is **Deluge**. Applications are assembled mainly through configuration, and code is written only where the logic is unusual.

A gym membership system is a good test case because it is a typical small-business data application: forms to capture data, reports to view it, and rules that run automatically (membership expiry, reminders, payment status).

3. Procedure

The comparison was carried out as a design analysis in these steps:

- Listed the functional requirements of the gym application.
- Broke the build into tasks for the traditional approach and for the Zoho Creator approach.
- Compared the two approaches on effort, skills, maintenance, security, cost, flexibility and scalability.
- Identified where each approach is the better fit and selected the approach for this project.

4. Requirements of the Gym Application

No.	Requirement	Example
R1	Register and store member details	Name, phone, DOB, photo, emergency contact
R2	Manage membership plans	Monthly, Quarterly, Annual with fees
R3	Subscribe a member to a plan	Auto-calculate end date from plan duration
R4	Record payments	Amount, mode, receipt number
R5	Track attendance	Check-in and check-out time
R6	Send reminders	Email 3 days before expiry
R7	Role-based access	Admin, Receptionist, Trainer, Member
R8	Dashboards and reports	Active members, expiring soon, monthly revenue

5. Task Breakdown by Approach

Requirement	Traditional development	Zoho Creator (low-code)
Data storage (R1-R5)	Design database schema, write migrations and an ORM layer	Create forms; tables and fields are generated automatically
Data entry screens	Build HTML/CSS/JS pages with validation code	Drag-and-drop form fields with built-in validation
Auto end date (R3)	Write server code and API endpoint	Deluge script in form workflow, a few lines
Email reminders (R6)	Set up SMTP service and a cron job	Scheduled workflow with the sendmail task
Login and roles (R7)	Implement authentication, sessions, permission checks	Built-in users, profiles and permission settings
Reports (R8)	Write queries and a charting front end	Configure list reports, charts and dashboard pages
Hosting and deployment	Provision servers, CI/CD, SSL	Hosted by the platform; publish with one action

6. Comparison

Criterion	Traditional	Low-code (Zoho Creator)
Development speed	Slow; weeks to months for a full app	Fast; a working prototype can be assembled in days
Skills needed	Programmers with front-end, back-end and database skills	Business users and junior developers with basic Deluge
Cost	High initial development and infrastructure cost	Subscription-based; lower build cost
Flexibility	Unlimited customisation	Limited to platform features; extensions via custom functions
Maintenance	Team must patch, upgrade and host	Platform vendor handles infrastructure and upgrades
Security	Developer is responsible for every control	Platform-level security; app roles still must be configured
Scalability	Can be engineered for very high load	Suitable for small to mid-size workloads; platform limits apply
Vendor lock-in	Low; code is owned and portable	Higher; app logic lives on the vendor platform
Integration	Any API with custom code	Ready connectors to Zoho apps; REST APIs through Deluge

7. Illustrative Code Comparison

Calculating a membership end date and marking it active illustrates the difference in effort. The snippets are illustrative design examples, not output from a deployed system.

Low-code (Deluge, form workflow):

```
plan = Membership_Plan[ID == input.Plan];
input.Start_Date = zoho.currentdate;
input.End_Date = input.Start_Date.addMonth(plan.Duration_Months);
input.Status = "Active";
```

Traditional (Node.js style outline):

```
app.post('/subscriptions', async (req, res) => {
  const plan = await db.plans.findById(req.body.planId);
  const start = new Date();
  const end = new Date(start);
  end.setMonth(end.getMonth() + plan.durationMonths);
  const sub = await db.subscriptions.create({
    memberId: req.body.memberId, planId: plan.id,
    startDate: start, endDate: end, status: 'Active' });
  res.status(201).json(sub);
});
// plus: routing, auth middleware, validation, DB setup, front-end form
```

8. Discussion

For the gym application the requirements are mostly forms, reports and simple automation, which are exactly the areas where low-code platforms are strongest. Zoho Creator removes the need to build login, hosting, database management and basic screens, so effort goes into the business rules instead.

Traditional development would be preferred if the gym chain needed a highly customised mobile experience, hardware integration such as biometric turnstiles at large scale, or full ownership of the code. Even then, a hybrid approach is possible: Zoho Creator for the administrative core, with custom functions or external services for the specialised parts.

Limitations of low-code that apply here are vendor dependence, platform limits on very large data volumes, and restricted control over the user interface. These are acceptable for a single-gym deployment.

9. Result

The analysis shows that low-code development with Zoho Creator is the more suitable approach for the Gym Membership Management application because it reduces build time, required skills and maintenance effort while still covering all identified requirements R1-R8. Traditional development remains the better choice only for highly specialised or very large-scale needs.

Evidence boundary: this record is a design and literature comparison. Effort and speed statements are qualitative and were not measured by building both versions.

10. References

- Zoho Creator product page: https://www.zoho.com/creator/
- Zoho Creator help documentation: https://www.zoho.com/creator/help/
- Deluge scripting language guide: https://www.zoho.com/deluge/help/

- Low-code development platform (overview):
  https://en.wikipedia.org/wiki/Low-code_development_platform